

图解大模型训练之：数据并行上篇(DP, DDP与ZeRO)

在上一篇的介绍中，我们介绍了以Google GPipe为代表的流水线并行范式。当模型太大，一块GPU放不下时，流水线并行将模型的不同层放到不同的GPU上，通过切割mini-batch实现对训练数据的流水线处理，提升GPU计算通讯比。同时通过re-materialization机制降低显存消耗。

但在实际应用中，流水线并行并不特别流行，主要原因是模型能否均匀切割，影响了整体计算效率，这就需要算法工程师做手调。因此，今天我们来介绍一种应用最广泛，最易于理解的并行范式：数据并行。

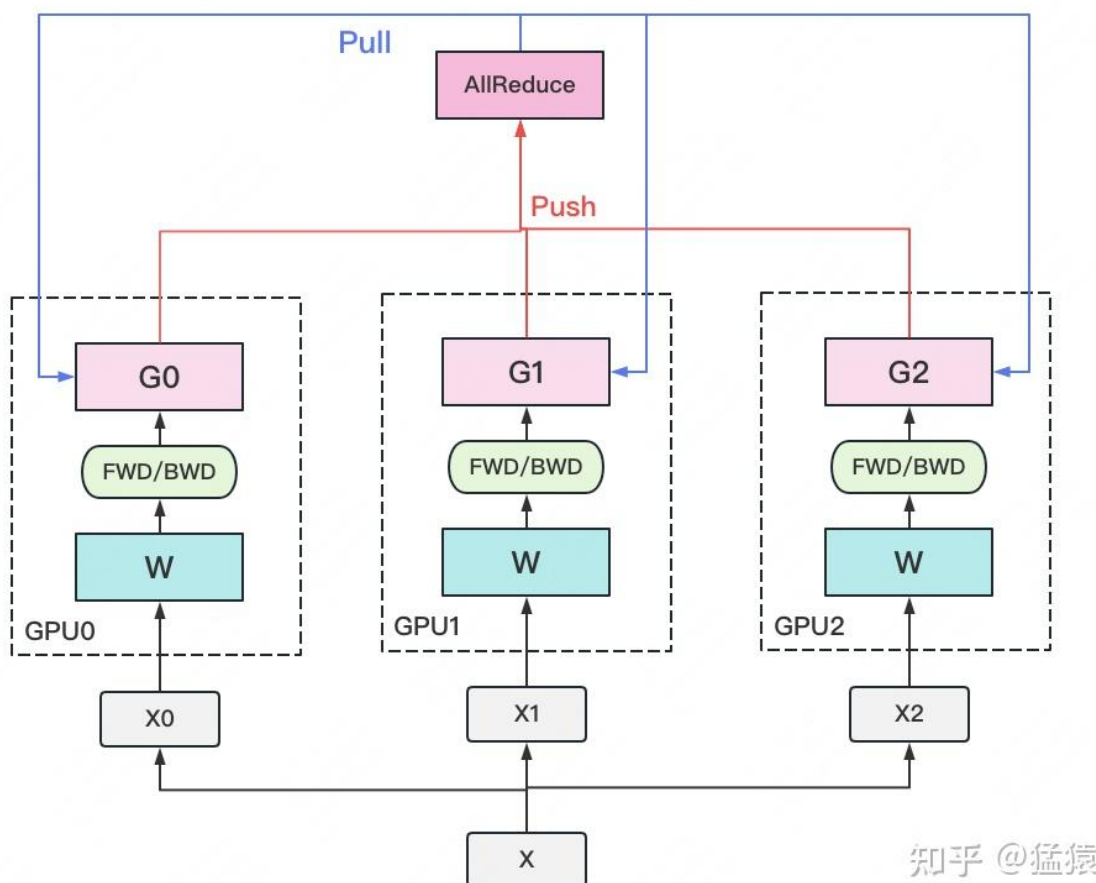
数据并行的核心思想是：在**各个GPU上都拷贝一份完整模型，各自吃一份数据，算一份梯度，最后对梯度进行累加来更新整体模型**。理念不复杂，但到了大模型场景，**巨大的存储和GPU间的通讯量**，就是系统设计要考虑的重点了。在本文中，我们将递进介绍三种主流数据并行的实现方式：

- **DP (Data Parallelism)**：最早的数据并行模式，一般采用[参数服务器](#)(Parameters Server)这一编程框架。实际中多用于单机多卡
- **DDP (Distributed Data Parallelism)**：分布式数据并行，采用Ring AllReduce的通讯方式，实际中多用于多机场景
- **ZeRO**：零冗余优化器。由微软推出并应用于其[DeepSpeed](#)框架中。严格来讲ZeRO采用数据并行+张量并行的方式，旨在降低存储。

本文将首先介绍DP和DDP，在下一篇文章里，介绍ZeRO。

一、数据并行 (DP)

1.1 整体架构



根据论文<https://web.eecs.umich.edu/~mosharaf/Readings/Parameter-Server.pdf>内容原创绘制

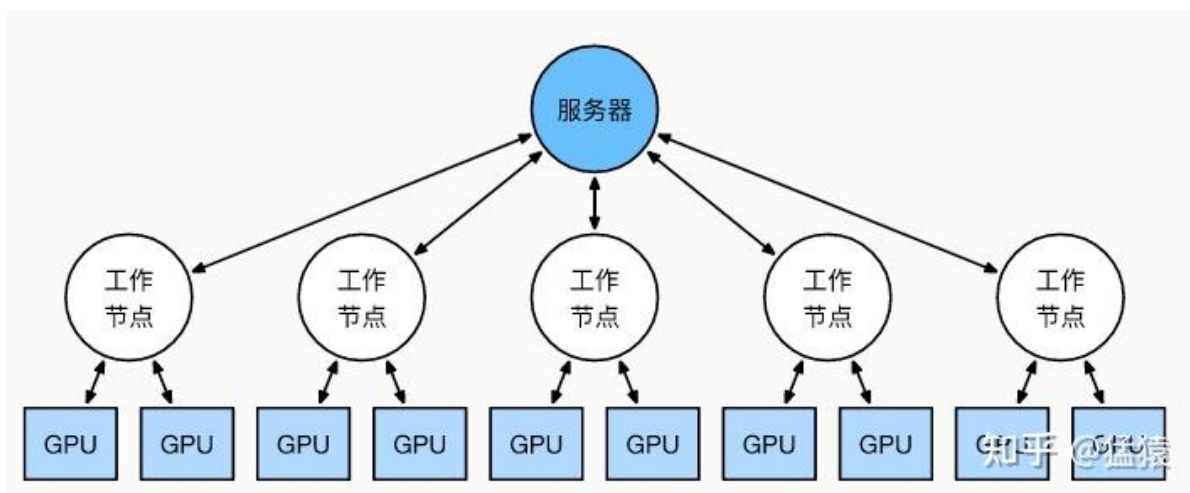
一个经典数据并行的过程如下：

- 若干块**计算GPU**，如图中GPU0~GPU2；1块**梯度收集GPU**，如图中AllReduce操作所在GPU。
- 在每块计算GPU上都拷贝一份完整的模型参数。
- 把一份数据X（例如一个batch）均匀分给不同的计算GPU。
- 每块计算GPU做一轮FWD和BWD后，算得一份梯度G。
- 每块计算GPU将自己的梯度**push**给梯度收集GPU，做聚合操作。这里的聚合操作一般指**梯度累加**。当然也支持用户自定义。
- 梯度收集GPU聚合完毕后，计算GPU从它那**pull**下完整的梯度结果，用于更新模型参数W。更新完毕后，计算GPU上的模型参数依然保持一致。
- **聚合再下发梯度的操作，称为AllReduce。**

前文说过，实现DP的一种经典编程框架叫“参数服务器”，在这个框架里，**计算GPU称为Worker**，**梯度聚合GPU称为Server**。在实际应用中，为了尽量减少通讯量，一般可选择一个Worker同时作为Server。比如可把梯度全发到GPU0上做聚合。需要再额外说明几点：

- 1个Worker或者Server下可以不止1块GPU。
- Server可以只做梯度聚合，也可以梯度聚合+全量参数更新一起做

在参数服务器的语言体系下，DP的过程又可以被描述下图：



图片来源: https://zh.d2l.ai/chapter_computational-performance/parameterserver.html

1.2 通讯瓶颈与梯度异步更新

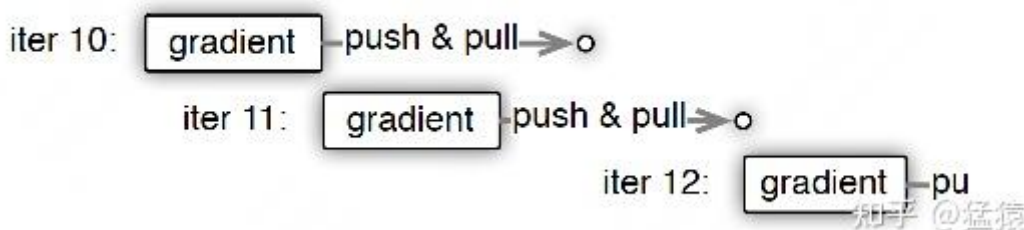
DP的框架理解起来不难，但实战中确有两个主要问题：

- **存储开销大。**每块GPU上都存了一份完整的模型，造成冗余。关于这一点的优化，我们将在后文ZeRO部分做讲解。
- **通讯开销大。**Server需要和每一个Worker进行梯度传输。当Server和Worker不在一台机器上时，Server的带宽将会成为整个系统的计算效率瓶颈。

我们对通讯开销再做详细说明。如果将传输比作一条马路，带宽就是马路的宽度，它决定每次并排行驶的数据量。例如带宽是100G/s，但每秒却推给Server 1000G的数据，消化肯定需要时间。那么当Server在搬运数据，计算梯度的时候，Worker们在干嘛呢？当然是在：

“聚众摸鱼~”

人类老板不愿意了：“打工系统里不允许有串行存在的任务！”，于是**梯度异步更新**这一管理层略诞生了。

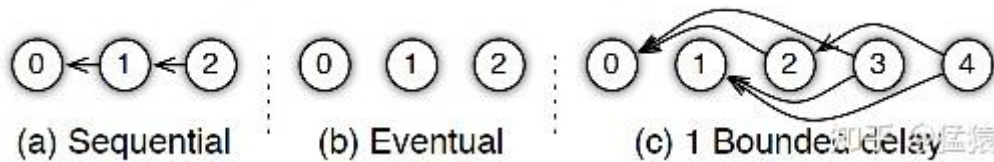


图片来源: <https://web.eecs.umich.edu/~mosharaf/Readings/Parameter-Server.pdf>

上图刻画了在**梯度异步更新**的场景下，某个Worker的计算顺序为：

- 在第10轮计算中，该Worker正常计算梯度，并向Server发送push&pull梯度请求。
- 但是，该Worker并不会实际等到把聚合梯度拿回来，更新完参数W后再做计算。而是直接拿旧的W，吃新的数据，继续第11轮的计算。**这样就保证在通讯的时间里，Worker也在马不停蹄做计算，提升计算通讯比。**
- 当然，异步也不能太过份。只计算梯度，不更新权重，那模型就无法收敛。图中刻画的是**延迟为1**的异步更新，也就是在开始第12轮对的计算时，必须保证W已经用第10、11轮的梯度做完2次更新了。

参数服务器的框架下，延迟的步数也可以由用户自己决定，下图分别刻划了几种延迟情况：



图片来源：<https://web.eecs.umich.edu/~mosharaf/Readings/Parameter-Server.pdf>

- (a) 无延迟
- (b) 延迟但不指定延迟步数。也即在迭代2时，用的可能是老权重，也可能是新权重，听天由命。
- (c) 延迟且指定延迟步数为1。例如做迭代3时，可以不拿回迭代2的梯度，但必须保证迭代0、1的梯度都已拿回且用于参数更新。

总结一下，异步很香，但对一个Worker来说，只是等于W不变，batch的数量增加了而已，在SGD下，会减慢模型的整体收敛速度。异步的整体思想是，比起让Worker闲着，倒不如让它多吃点数据，虽然反馈延迟了，但只要它在干活在学习就行。

batch就像活，异步就像画出去的饼，且往往不指定延迟步数，每个Worker干越来越多的活，但模型却没收敛取效，这又是刺伤了哪些打工仔们的心（狗头

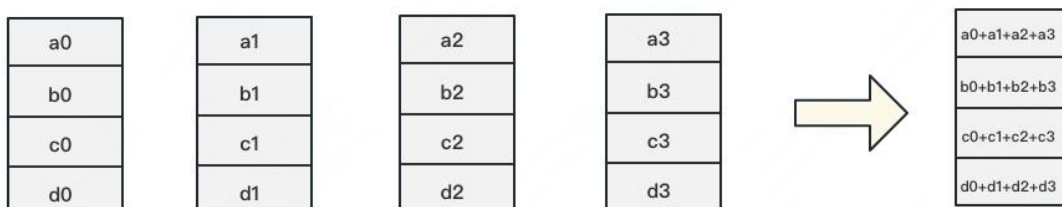
二、分布式数据并行(DDP)

受通讯负载不均的影响，DP一般用于单机多卡场景。因此，DDP作为一种更通用的解决方案出现了，既能多机，也能单机。DDP首先要解决的就是通讯问题：将Server上的通讯压力均衡转到各个Worker上。实现这一点后，可以进一步去Server，留Worker。

前文我们说过，聚合梯度 + 下发梯度这一轮操作，称为AllReduce。接下来我们介绍目前最通用的AllReduce方法：Ring-AllReduce。它由百度最先提出，非常有效地解决了数据并行中通讯负载不均的问题，使得DDP得以实现。

2.1 Ring-AllReduce

如下图，假设有4块GPU，每块GPU上的数据也对应被切成4份。AllReduce的最终目标，就是让每块GPU上的数据都变成箭头右边汇总的样子。



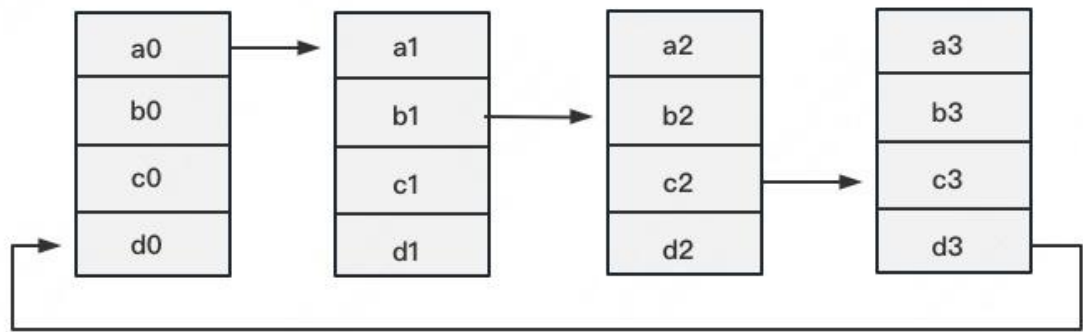
知乎 @猛猿

根据<https://blog.csdn.net/dpppBR/article/details/80445569>介绍原创绘制

Ring-ALLReduce则分两大步骤实现该目标：Reduce-Scatter和All-Gather。

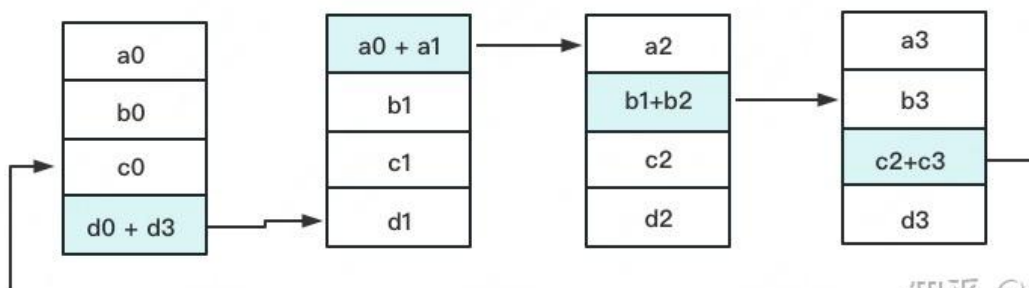
Reduce-Scatter

定义网络拓扑关系，使得每个GPU只和其相邻的两块GPU通讯。每次发送对应位置的数据进行累加。每一次累加更新都形成一个拓扑环，因此被称为Ring。看到这觉得困惑不要紧，我们用图例把详细步骤画出来。



知乎 @猛猿

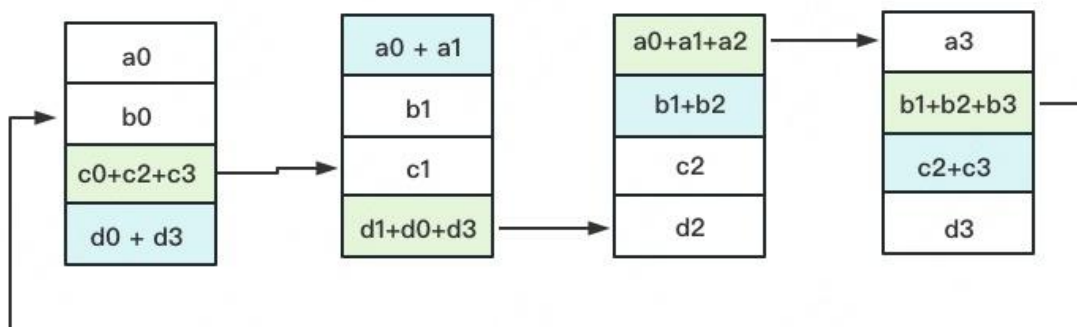
根据<https://blog.csdn.net/dpppBR/article/details/80445569>介绍原创绘制



知乎 @猛猿

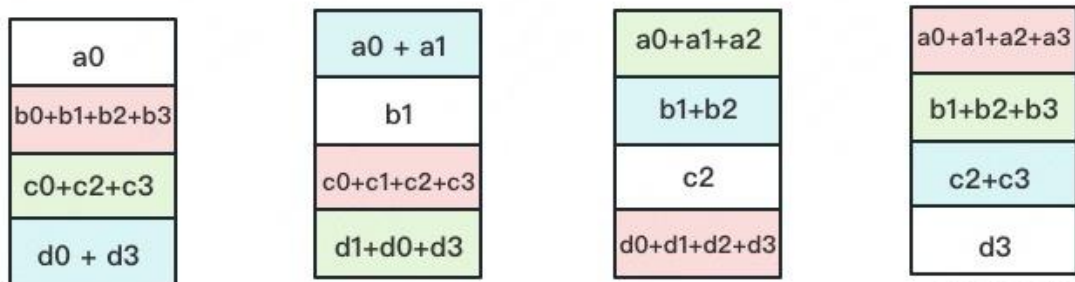
根据<https://blog.csdn.net/dpppBR/article/details/80445569>介绍原创绘制

一次累加完毕后，蓝色位置的数据块被更新，被更新的数据块将成为下一次更新的起点，继续做累加操作。



知乎 @猛猿

根据<https://blog.csdn.net/dpppBR/article/details/80445569>介绍原创绘制



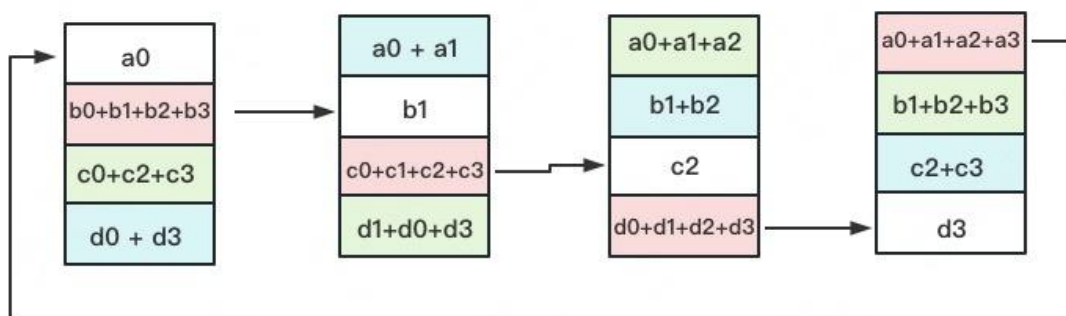
知乎 @猛猿

根据<https://blog.csdn.net/dpppBR/article/details/80445569>介绍原创绘制

3次更新之后，每块GPU上都有一块数据拥有了对应位置完整的聚合（图中红色）。此时，Reduce-Scatter阶段结束。进入All-Gather阶段。目标是把红色块的数据广播到其余GPU对应的位置上。

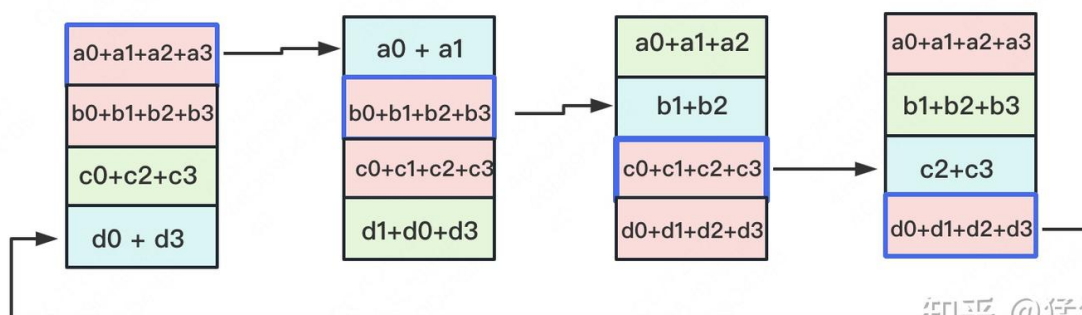
All-Gather

如名字里Gather所述的一样，这操作里依然按照“相邻GPU对应位置进行通讯”的原则，但对对应位置数据不再做相加，而是直接替换。All-Gather以红色块作为起点。



知乎 @猛猿

根据<https://blog.csdn.net/dpppBR/article/details/80445569>介绍原创绘制



知乎 @猛猿

根据<https://blog.csdn.net/dpppBR/article/details/80445569>介绍原创绘制

以此类推，同样经过**3轮迭代后**，使得每块GPU上都汇总到了完整的数据，变成如下形式：

$a_0+a_1+a_2+a_3$
$b_0+b_1+b_2+b_3$
$c_0+c_1+c_2+c_3$
$d_0+d_1+d_2+d_3$

根据<https://blog.csdn.net/dpppBR/article/details/80445569>介绍原创绘制

建议读者们手动推一次，加深理解。

2.2 Ring-AllReduce通讯量分析

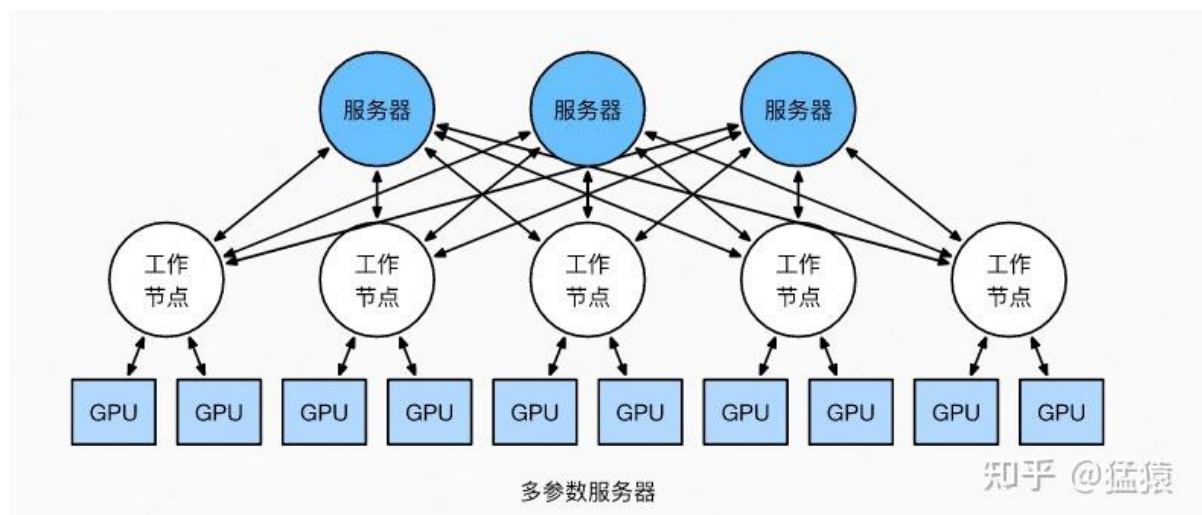
假设模型参数 W 的大小为 W ，GPU个数为 N 。则梯度大小也为 W ，每个梯度块的大小为 W/N 。对单卡GPU来说（只算其send通讯量）：

- Reduce-Scatter阶段，通讯量为 W/N
- All-Gather阶段，通讯量为 W

单卡总通讯量为 W ，随着 N 的增大，可以近似为 W 。全卡总通讯量为 NW

而对前文的DP来说，它的Server承载的通讯量是 W ，Workers为 N ，全卡总通讯量依然为 NW 。虽然通讯量相同，但搬运相同数据量的时间却不一定相同。DDP把通讯量均衡负载到了每一时刻的每个Worker上，而DP仅让Server做勤劳的搬运工。当越来越多的GPU分布在距离较远的机器上时，DP的通讯时间是会增加的。

但这并不说明参数服务器不能打（有很多文章将参数服务器当作old dinosaur来看）。事实上，参数服务器也提供了多Server方法，如下图：



在多Server的模式下，进一步，每个Server可以只负责维护和更新某一块梯度（也可以某块梯度+参数一起维护），此时虽然每个Server仍然需要和所有Worker通讯，但它的带宽压力会小非常多。经过调整后，依然可以用来做DDP。虽然这篇文章是用递进式的方式来介绍两者，但不代表两者间一定要决出优劣。我想表达的观点是，方法是多样性的。对参数服务器有兴趣的朋友，可以阅读参考的第1个链接。

最后，请大家记住Ring-AllReduce的方法，因为在之后的ZeRO，Megatron-LM中，它将频繁地出现，是分布式训练系统中重要的算子。

三、总结

- 1、在DP中，每个GPU上都拷贝一份完整的模型，每个GPU上处理batch的一部分数据，所有GPU算出来的梯度进行累加后，再传回各GPU用于更新参数
- 2、DP多采用参数服务器这一编程框架，一般由若个计算Worker和1个梯度聚合Server组成。Server与每个Worker通讯，Worker间并不通讯。因此Server承担了系统所有的通讯压力。基于此DP常用于单机多卡场景。
- 3、异步梯度更新是提升计算通讯比的一种方法，延迟更新的步数大小决定了模型的收敛速度。
- 4、Ring-AllReduce通过定义网络环拓扑的方式，将通讯压力均衡地分到每个GPU上，使得跨机器的数据并行（DDP）得以高效实现。
- 5、DP和DDP的总通讯量相同，但因负载不均的原因，DP需要耗费更多的时间搬运数据

四、参考

- 1、<https://web.eecs.umich.edu/~mosharaf/Readings/Parameter-Server.pdf>
- 2、https://zh.d2l.ai/chapter_computational-performance/parameterserver.html
- 3、<https://blog.csdn.net/dpppBR/article/details/80445569>
- 4、<https://arxiv.org/abs/1910.02054>
- 5、https://blog.51cto.com/u_146917

图解大模型训练之：数据并行下篇(DeepSpeed ZeRO，零冗余优化)

在数据并行上篇中，我们介绍了朴素数据并行（DP）与分布式数据并行（DDP）。两者的总通讯量虽然相同，但DP存在负载不均的情况，大部分的通讯压力集中在Server上，而Server的通讯量与GPU数量呈线性关系，导致DP一般适用于**单机多卡**场景。而DDP通过采用Ring-AllReduce这一NCCL操作，使得通讯量均衡分布到每块GPU上，且该通讯量为一个固定常量，不受GPU个数影响，因此可实现**跨机器**的训练。（[阅读本文前，强烈建议阅读上一篇](#)）

在上篇介绍中，**通讯负载不均**的优化我们解释过了，但还遗留了一个**显存开销**问题：数据并行中，每个GPU上都复制了一份完整模型，当模型变大时，很容易打爆GPU的显存，那要怎么办呢？

今天这篇文章，我们将介绍由微软开发的**ZeRO（零冗余优化）**，它是DeepSpeed这一分布式训练框架的核心，被用来解决大模型训练中的显存开销问题。**ZeRO的思想就是用通讯换显存**。如果初读ZeRO，觉得它逻辑跳跃，晦涩难懂，那么这篇文章或许可以帮到你～

【20231205更新】

针对评论区问得比较多的问题，这边做一下回复：

(1) stage1的通讯量为什么是而不是？

先说结论：实操中是，按论文概念定义是。

在实操中，我们可以只对梯度做一次scatter-reduce，并用各自维护的optimizer去更新对应的W，然后再对W做all-gather使得每块卡上都有更新后的完整W，这样通讯量就是。

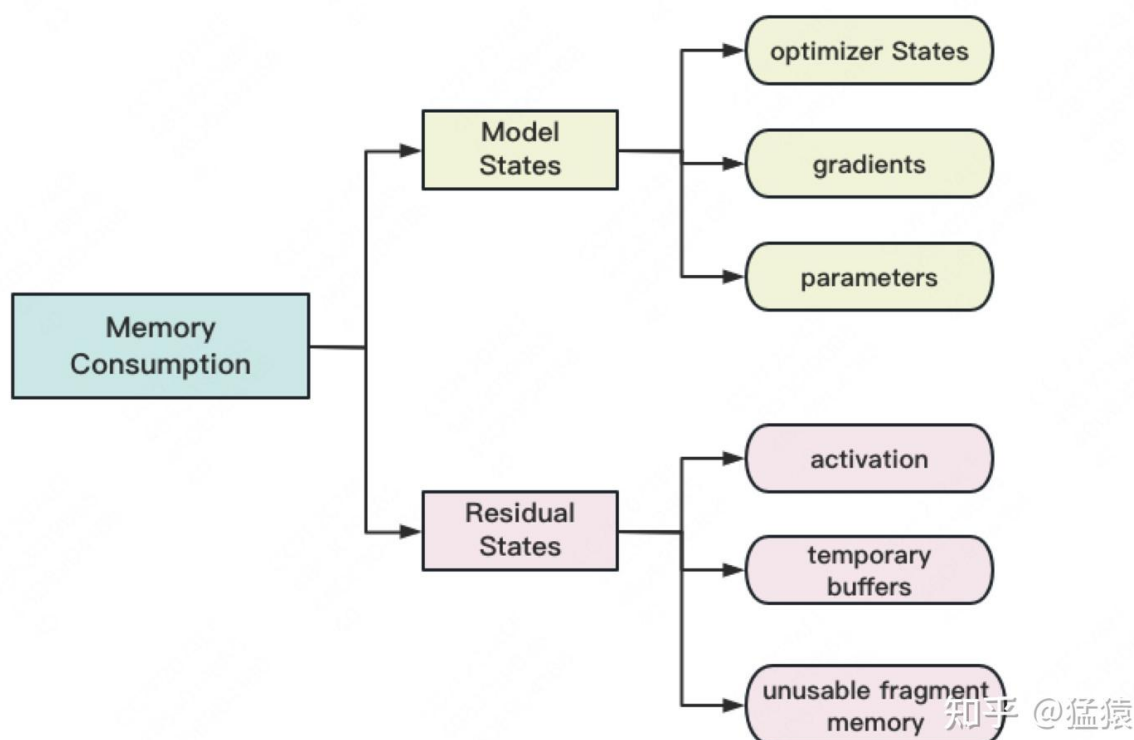
那么 是怎么来的呢？因为论文定义stage1只有optimizer是切开的，意味着G和W都是完整的。所以对G做all-reduce（虽然拿回完整的G并没有意义），对W做all-gather，这样通讯量就是。

本文写作时，最终选择按照论文对相关概念的定义，选择了，但是实操来看是完全可以用 实现的。评论区有朋友提到deepspeed的某次代码更新是将stage1的通讯量从 降至，可能也是基于此做了改进。

一、存储消耗

1.1 存储分类

首先，我们来看在大模型训练的过程中，GPU都需要存什么内容。



存储主要分为两大块：Model States和Residual States

Model States指和模型本身息息相关的，必须存储的内容，具体包括：

- **optimizer states**: [Adam优化](#)算法中的momentum和variance
- **gradients**: 模型梯度
- **parameters**: 模型参数W

Residual States指并非模型必须的，但在训练过程中会额外产生的内容，具体包括：

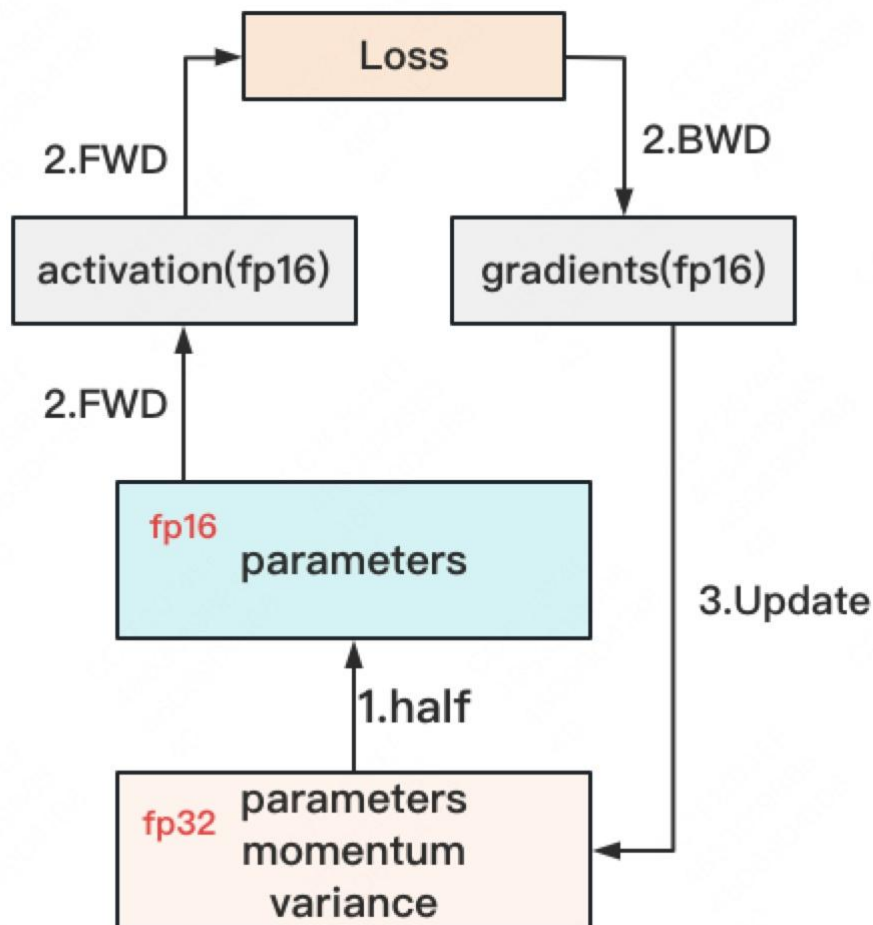
- **activation**: 激活值。在流水线并行中我们曾详细介绍过。在backward过程中使用链式法则计算梯度时会用到。有了它算梯度会更快，但它不是必须存储的，因为可以通过重新做Forward来算它。
- **temporary buffers**: 临时存储。例如把梯度发送到某块GPU上做加总聚合时产生的存储。
- **unusable fragment memory**: 碎片化的存储空间。虽然总存储空间是够的，但是如果取不到连续的存储空间，相关的请求也会被fail掉。对这类空间浪费可以通过内存整理来解决。

1.2 精度混合训练

知道了存储分类，进一步，我们想知道，假设模型的参数 W 大小是 Φ ，那么每一类存储具体占了多大的空间呢？

在分析这个问题前，我们需要来了解**精度混合训练**。

对于模型，我们肯定希望其参数越精准越好，也即我们用**fp32（单精度浮点数，存储占4byte）**来表示参数 W 。但是在forward和backward的过程中，fp32的计算开销也是庞大的。那么能否在计算的过程中，引入**fp16或bf16（半精度浮点数，存储占2byte）**，来减轻计算压力呢？于是，**混合精度训练**就产生了，它的步骤如下图：



知乎 @猛猿

- 存储一份fp32的parameter，momentum和variance（统称model states）
- 在forward开始之前，额外开辟一块存储空间，将fp32 parameter减半到fp16 parameter。
- 正常做forward和backward，在此之间产生的activation和gradients，都用fp16进行存储。
- 用fp16 gradients去更新fp32下的model states。
- 当模型收敛后，fp32的parameter就是最终的参数输出。

通过这种方式，混合精度训练在计算开销和模型精度上做了权衡。如果不了解fp32，fp16和bf16的细节也没关系，不影响下文的阅读。只要记住它们所占的存储空间和精度表达上的差异即可。

1.3 存储大小

现在，我们可以来计算模型在训练时需要的存储大小了，假设模型的参数 W 大小是，**以byte为单位**，存储如下：

	分类	大小	大小总计
必存	parameter (fp32)	4Φ	12Φ
	momentum(fp32)	4Φ	
	variance (fp32)	4Φ	
中间值	parameter (fp16)	2Φ	4Φ
	gradients (fp15)	2Φ	
总计			16Φ

知乎 @猛猿

因为采用了Adam优化，所以才会出现momentum和variance，当然你也可以选择别的优化办法。因此这里为了更通用些，记模型必存的数据大小为。因此最终内存开销为：

另外，**这里暂不将activation纳入统计范围**，原因是：

- activation不仅与模型参数相关，还与batch size相关
- activation的存储不是必须的。存储activation只是为了在用链式法则做backward的过程中，计算梯度更快一些。但你永远可以通过只保留最初的输入X，重新做forward来得到每一层的activation（虽然实际中并不会这么极端）。
- 因为activation的这种灵活性，纳入它后不方便衡量系统性能随模型增大的真实变动情况。因此在这里不考虑它，在后面会单开一块说明对activation的优化。

二、ZeRO-DP

知道了什么东西会占存储，以及它们占了多大的存储之后，我们就可以来谈如何优化存储了。

注意到，在整个训练中，有很多states并不会每时每刻都用到，举例来说；

- Adam优化下的optimizer states只在最终做update时才用到
- 数据并行中，gradients只在最后做AllReduce和updates时才用到
- 参数W只在forward和backward的那一刻才用到
- 诸如此类

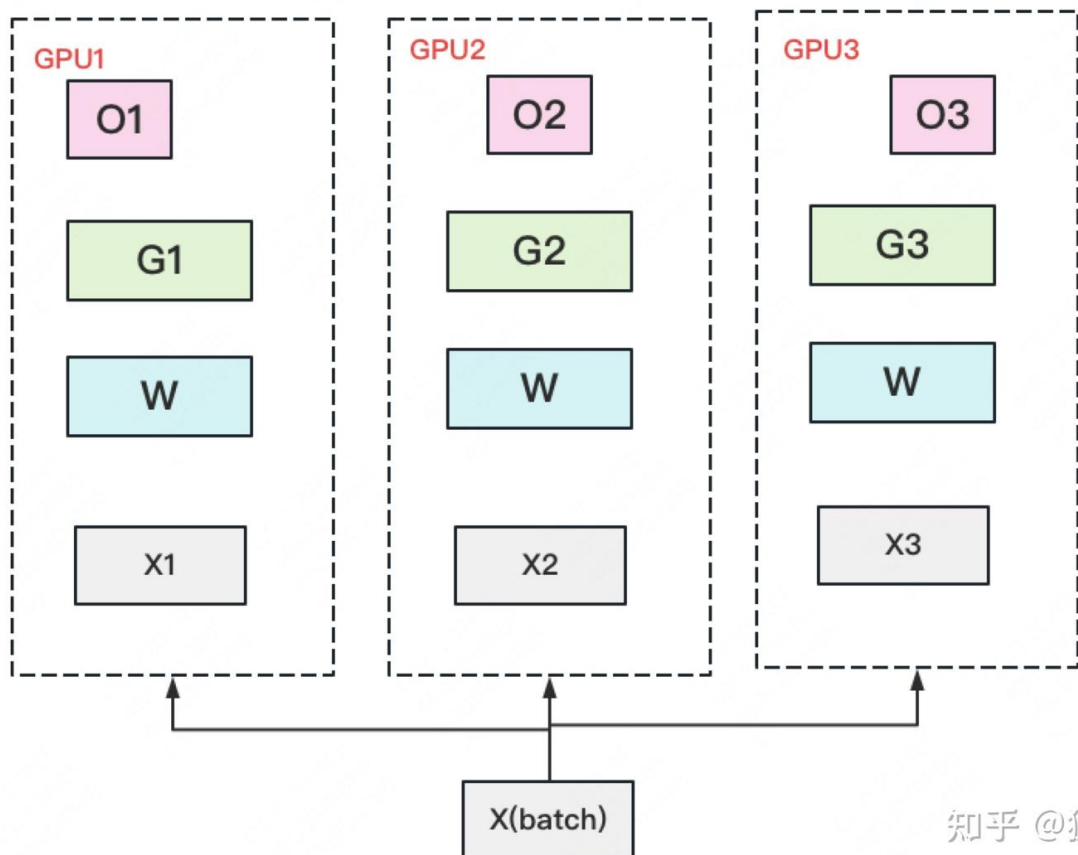
所以，ZeRO想了一个简单粗暴的办法：**如果数据算完即废，等需要的时候，我再想办法从个什么地方拿回来，那不就省了一笔存储空间吗？**

沿着这个思路，我们逐一起来看ZeRO是如何递进做存储优化的。

(阅读前，如果不熟悉GPU间数据通讯操作，一定要先读[猛猿：图解大模型训练之：数据并行上篇\(DP, DDP与ZeRO\)这一篇](#))

2.1：优化状态分割

首先，从 optimizer state开始优化。将optimizer state分成若干份，每块GPU上各自维护一份。这样就减少了相当一部分的显存开销。如下图：



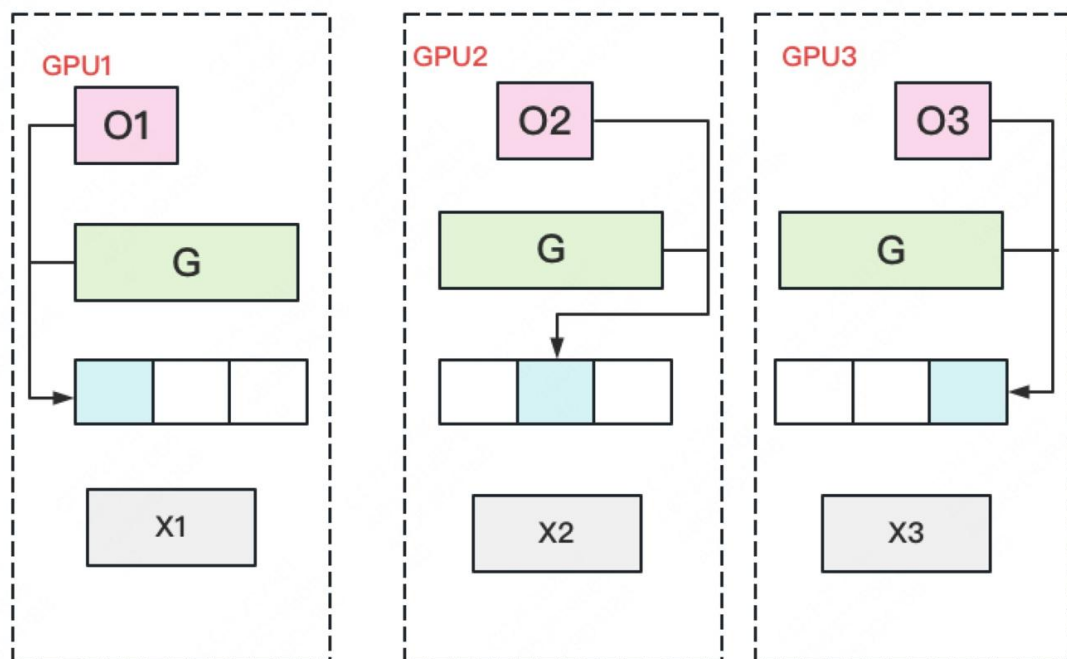
知乎 @猛猿

复习一下，此时 $W=fp16$ ， $G=fp16$ ， $O=fp32$ 。此时，整体数据并行的流程如下：

(1) 每块GPU上存一份完整的参数 W 。将一个batch的数据分成3份，每块GPU各吃一份，做完一轮 forward和backward后，各得一份梯度。

(2) 对梯度做一次AllReduce，得到完整的梯度 G ，产生单卡通讯量。为了表达简明，这里通讯量我们就不再换算成byte了，而直接根据参数量来计算。对AllReduce (reduce-scatter + all-gather) 不熟悉的朋友，可以先去看上一篇文章（PS：在zero1代码早期实践中，这里对梯度使用的是AllReduce，但是你可以发现，其实只要做一次ReduceScatter就完全够用了，产生的通讯量为）

(3) 得到完整梯度 G ，就可以对权重进行更新。我们知道 W 的更新由optimizer states和梯度共同决定。由于每块GPU上只保管部分optimizer states，只能将相应的 W （蓝色部分）进行更新。（2）和（3）可以用下图表示：



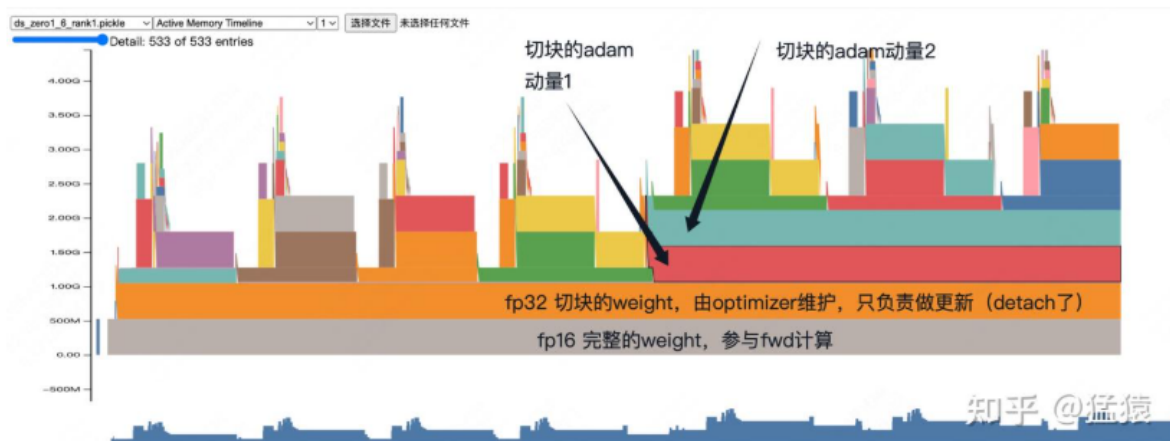
知乎 @猛猿

在混合精度训练的前提下，这边我们对【只能将相应的W（蓝色部分）进行更新】这句话做更严谨的说明，感谢

[@Chayenne Zhao](#)

在这里帮忙指出。

实际上，zero1并不是直接更新蓝色权重（fp16），而是直接更新红色优化器中维护的权重(fp32)，蓝色权重是由红色权重cast而来（参见本文1.2节）。这边我写了一个2卡zero1的toy example，并使用pytorch memory snapshot跟踪记录了它的显存使用信息，然后沿着记录结果中的堆栈信息，帮大家更好追溯这个过程：



知乎 @猛猿

(1) 完整的模型大小是1000MB（fp32精度下）

(2) fp16: zero会开辟500MB显存，存放完整的fp16模型（每张卡上都有完整的fp16），这部分权重负责实际执行

fwd和bwd过程

(3) fp32: `fp32 = partiton_fp16.copy().float().detach()`，
即fp16模型切块，转fp32，放在optimizer中，占据500MB显存

这部分权重从计算图中detach，只负责做更新

(4) 每次【成功】梯度更新后，单卡上`fp16._copy(fp32)`，再做`all-gather`。

因为是原位copy，不需要分配新显存，所以你会看到那个灰色显存块从头到尾都在。

【不成功】的梯度更新指：

梯度出现nan/inf，本轮权重不更新（如图中前3个step）

第4个step梯度正常，可以更新，这时候才会触发adam优化器2个动量的初始化

(4) 此时，每块GPU上都有部分W没有完成更新（图中白色部分）。所以我们需要对W做一次All-Gather（此时的W已经执行过上述的 `fp16._copy(fp32)` 步骤），从别的GPU上把更新好的部分W取回来。产生单卡通讯量。

做完后，设GPU个数为，显存和通讯量的情况如下：

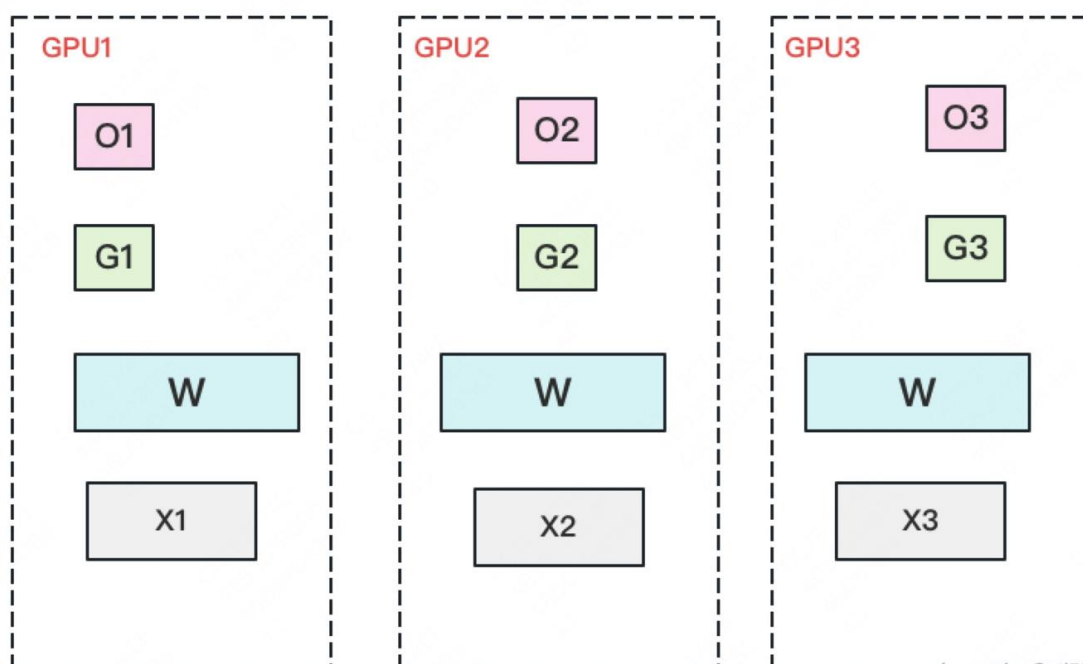
	显存	显存（单位GB） $K = 12, \Phi = 7.5B, N_d = 64$	单卡通讯量
朴素DP	$(2 + 2 + K)\Phi$	120GB	2Φ
P_{os}	$(2 + 2 + \frac{K}{N_d})\Phi$	31.4GB	3Φ

知乎 @猛猿

假设各变量大小如表格第二列所示，那么在增加1.5倍单卡通讯开销的基础上，将单卡存储降低了4倍。看起来是个还不错的trade-off，那么还能做得更好吗

2.2：优化状态与梯度分割

现在，更进一步，我们把梯度也拆开，每个GPU格子维护一块梯度。

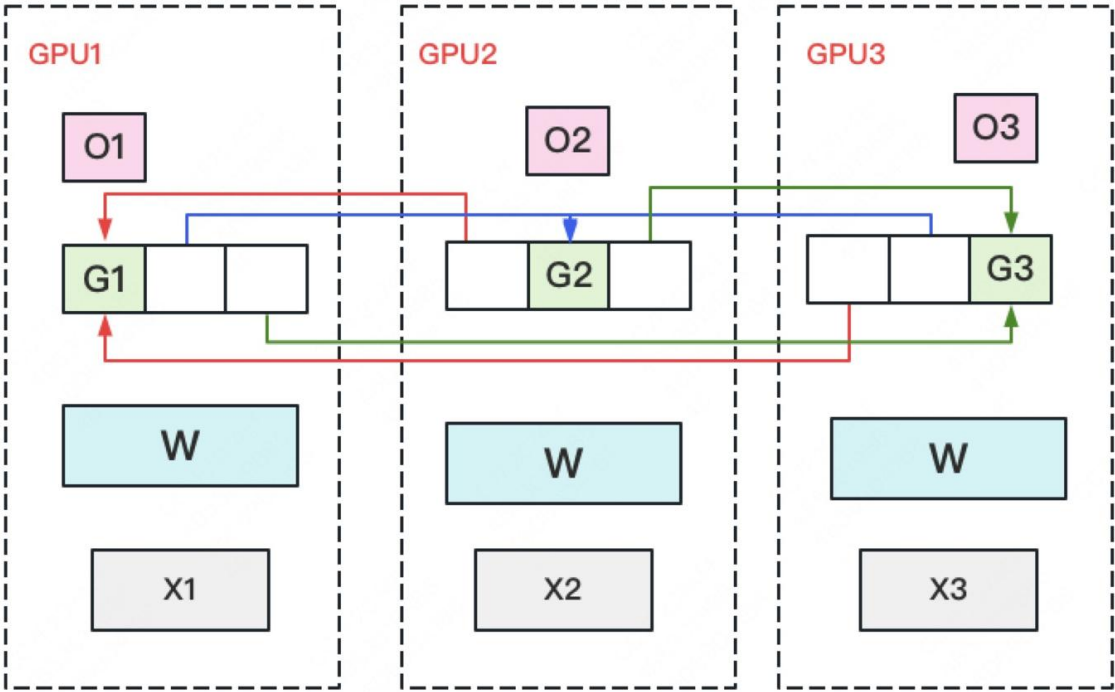


知乎 @猛猿

此时，数据并行的整体流程如下：

(1) 每块GPU上存一份完整的参数W。将一个batch的数据分成3份，每块GPU各吃一份，做完一轮forward和backward后，**算得一份完整的梯度（下图中绿色+白色）**。

(2) 对梯度做一次**Reduce-Scatter**，保证每个GPU上所维持的那块梯度是聚合梯度。例如对GPU1，它负责维护G1，因此其他的GPU只需要把G1对应位置的梯度发给GPU1做加总就可。汇总完毕后，白色块对GPU无用，可以从显存中移除。单卡通讯量。（1）和（2）见下图：



做完对应梯度加总后，白色梯度可以移除，绿色梯度为最终汇总梯度

知乎 @猛猿

(3) 每块GPU用自己对应的O和G去更新相应的W。更新完毕后，**每块GPU维持了一块更新完毕的W**。同理，对W做一次**All-Gather**，将别的GPU算好的W同步到自己这来。单卡通讯量。

再次比对下显存和通讯量：

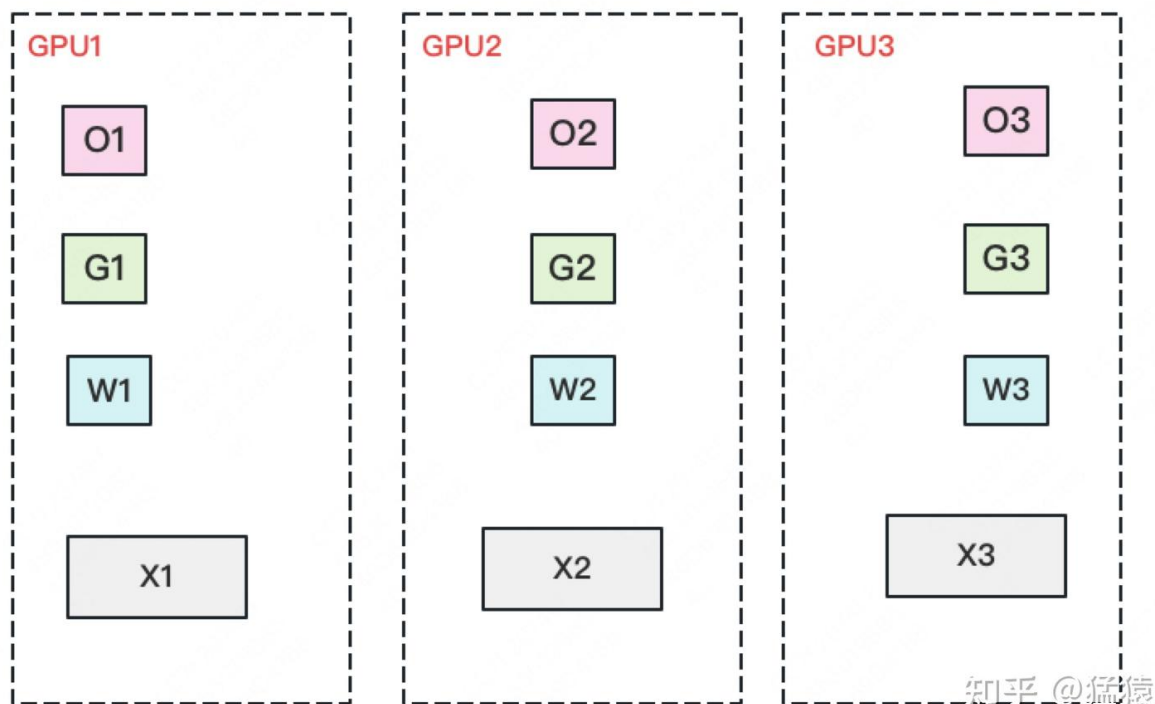
	显存	显存（单位GB） $K = 12, \Phi = 7.5B, N_d = 64$	单卡通讯量
朴素DP	$(2 + 2 + K)\Phi$	120GB	2Φ
P_{os}	$(2 + 2 + \frac{K}{N_d})\Phi$	31.4GB	3Φ
$P_{os} + P_g$	$(2 + \frac{2 + K}{N_d})\Phi$	16.6GB	2Φ

知乎 @猛猿

和朴素DP相比，**存储降了8倍，单卡通讯量持平**，好像更牛皮了呢！那么，还可以优化吗？

2.3：优化状态、梯度与参数分割

看到这里，也许你有点感觉了，**ZeRO的思想就是：万物皆可切，万物皆可抛。**所以现在，我们把参数也切开。每块GPU置维持对应的optimizer states, gradients和parameters（即W）。



数据并行的流程如下：

- (1) 每块GPU上只保存部分参数W。将一个batch的数据分成3份，每块GPU各吃一份。
- (2) 做forward时，对W做一次**All-Gather**，取回分布在别的GPU上的W，得到一份完整的W，单卡通讯量。**forward做完，立刻把不是自己维护的W抛弃。**
- (3) 做backward时，对W做一次**All-Gather**，取回完整的W，单卡通讯量。**backward做完，立刻把不是自己维护的W抛弃。**
- (4) 做完backward，算得一份完整的梯度G，对G做一次**Reduce-Scatter**，从别的GPU上聚合自己维护的那部分梯度，单卡通讯量。**聚合操作结束后，立刻把不是自己维护的G抛弃。**
- (5) 用自己维护的O和G，更新W。由于只维护部分W，因此无需再对W做任何AllReduce操作。

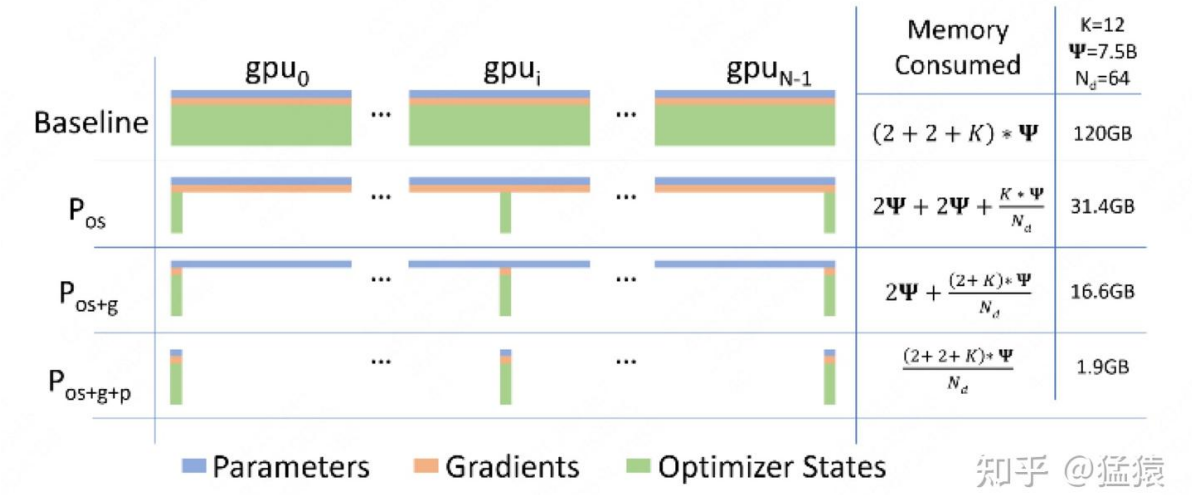
显存和通讯量如下：

	显存	显存（单位GB） $K = 12, \Phi = 7.5B, N_d = 64$	单卡通讯量
朴素DP	$(2 + 2 + K)\Phi$	120GB	2Φ
P_{os}	$(2 + 2 + \frac{K}{N_d})\Phi$	31.4GB	3Φ
$P_{os} + P_g$	$(2 + \frac{2 + K}{N_d})\Phi$	16.6GB	2Φ
$P_{os} + P_g + P_p$	$(\frac{2 + 2 + K}{N_d})\Phi$	1.9GB	3Φ

知乎 @猛猿

到这一步，我们用1.5倍的通讯开销，换回近120倍的显存。只要梯度计算和异步更新做的好，通讯时间大部分可以被计算时间隐藏，因此这样的额外通讯开销，也是划算的。

到这里，我们可以放出原始论文中的说明图了，经过以上分析，这张说明图是不是瞬间就能看懂了。不得不吐槽下，虽然ZeRO的设计不复杂，但对应论文写得真是逻辑跳跃，晦涩难懂....



仔细一想，ZeRO其实掌握了降本增效的精髓：用完即弃，需要再补。反正我补一个和你差不多的，也不会花费很多通（找）讯（人）时间，还大大降低了我的成本。模型的每一层多算（造）几（轮）遍（子）有啥关系呢，反正在我的预算里每个人都一刻不停地干活，就行啦！

2.4 ZeRO VS 模型并行

知道模型并行的朋友，可能会想，既然ZeRO都把参数W给切了，那它应该是个模型并行呀？为什么要归到数据并行里呢？

其实ZeRO是模型并行的形式，数据并行的实质。
模型并行，是指在forward和backward的过程中，我只需要用自己维护的那块W来计算就行。即**同样的输入X，每块GPU上各算模型的一部分，最后通过某些方式聚合结果。**
但对ZeRO来说，它做forward和backward的时候，是需要把各GPU上维护的W聚合起来的，即本质上还是用完整的W进行计算。**它是不同的输入X，完整的参数W，最终再做聚合。**
因为下一篇要写模型并行Megatron-LM，因此现在这里罗列一下两者的对比。

三、ZeRO-R

说完了以上对model states的显存优化，现在来看对residual states的优化。

3.1 : Partitioned Activation Checkpointing

前面说过，对activation的存储是灵活的。不像optimizer states，gradients和parameters对模型更新是必须的，activation只是起到加速梯度计算的作用。因此，在哪几层保存activation，保存哪些activation都是可以灵活设置的。同样，我们也可以仿照以上切割方式，每块GPU上只维护部分的activation，需要时再从别的地方聚合过来就行。需要注意的是，activation对显存的占用一般会远高于模型本身，通讯量也是巨大的，所以这块要灵活、有效地实验设计。

3.2 : Constant Size Buffer

固定大小的内存buffer，它的目的在于：

- 提升带宽利用率。当GPU数量上升，GPU间的通讯次数也上升，每次的通讯量可能下降（但总通讯量不会变）。数据切片小了，就不能很好利用带宽了。所以这个buffer起到了积攒数据的作用：等数据积攒到一定大小，再进行通讯。
- 使得存储大小可控。在每次通讯前，积攒的存储大小是常量，是已知可控的。更方便使用者对训练中的存储消耗和通讯时间进行预估。

3.3 : Memory Defragmentation

在前文提过，设置机制，对碎片化的存储空间进行重新整合，整出连续的存储空间。防止出现总存储足够，但连续存储不够而引起的存储请求fail

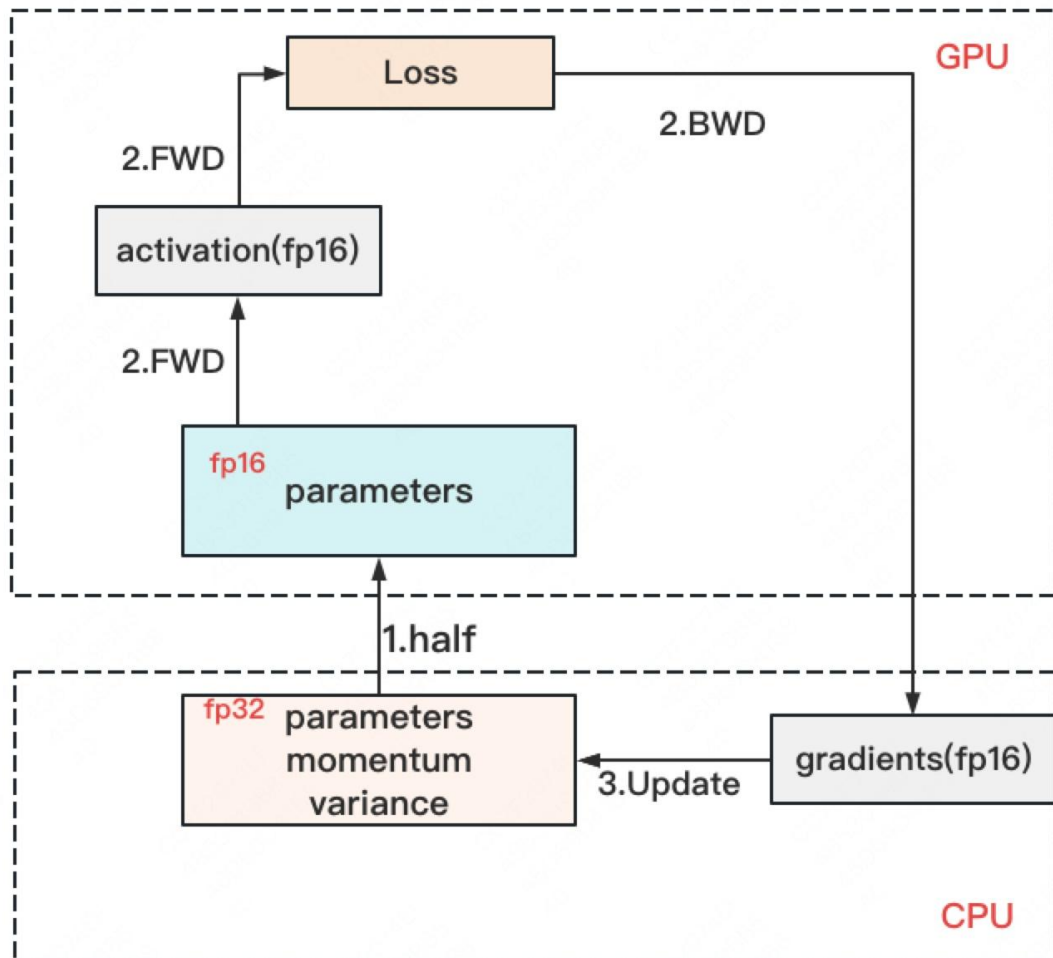
四、[ZeRO-Offload](#)与[ZeRO-Infinity](#)

最后，简单介绍一下ZeRO-Offload。它的核心思想是：**显存不够，内存来凑**。如果我把要存储的大头卸载(offload)到CPU上，而把计算部分放到GPU上，**这样比起跨机，是不是能既降显存，也能减少一些通讯压力呢？**

ZeRO-Offload的做法是：

- **forward和backward计算量高**，因此和它们相关的部分，例如参数W (fp16) , activation, 就全放入GPU。
- **update的部分计算量低**，因此和它相关的部分，全部放入CPU中。例如W(fp32), optimizer states (fp32) 和gradients(fp16)等。

具体切分如下图：



知乎 @猛猿

ZeRO-infinity也是同理，它们在解决的事情都是：找个除GPU之外的地方，存数据。感兴趣的朋友可以深入研究，这里就不展开了。

五、参考

- 1、<https://arxiv.org/pdf/1910.02054.pdf>
- 2、https://blog.51cto.com/u_14691718/5631471
- 3、https://blog.csdn.net/qg_433070